



Compiler Construction

Lab 8

Name: Fatima Zahra

Sap Id: 55551

Submitted To: Ma'am Hasnat

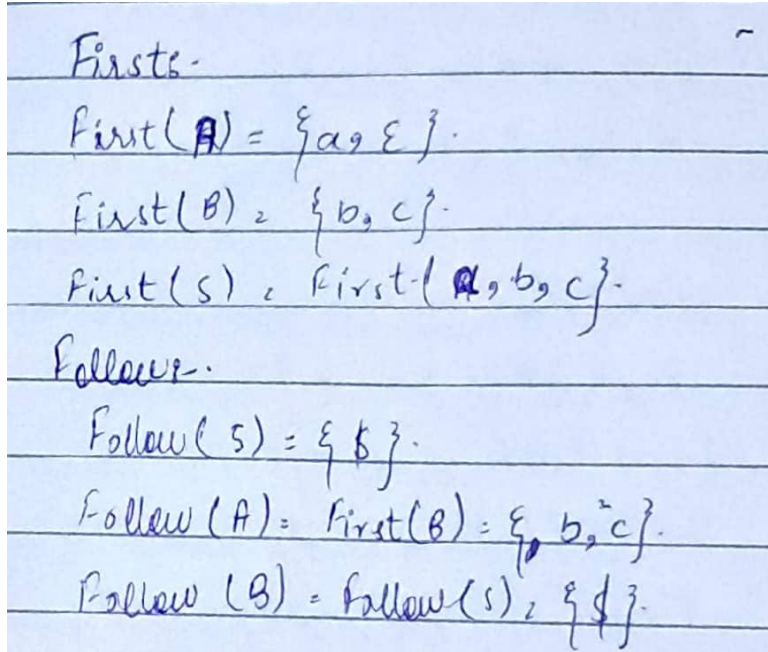
Riphah International University of Islamabad

Task 1:

$S \rightarrow A B$

$A \rightarrow a A \mid \epsilon$

$B \rightarrow b B \mid c$



Code:

// Task1.cpp - Simple FIRST and FOLLOW for Grammar 1

```
#include <iostream>
```

```
#include <map>
```

```
#include <set>
```

```
#include <string>
```

```
using namespace std;
```

```
void printSet(string name, set<char> s) {
```

```
    cout << name << " = { ";
```

```
    for (set<char>::iterator it = s.begin(); it != s.end(); ++it) {
```

```
        if (*it == '#') cout << "e ";
```

```
        else cout << *it << " ";
```

```
    }
```

```
    cout << "}\n";
```

```
}
```

```
int main() {  
    cout << "GRAMMAR 1:\n";  
    cout << "S -> A B\n";  
    cout << "A -> a A | e\n";  
    cout << "B -> b B | c\n\n";  
  
    map<string, set<char> > first;  
    map<string, set<char> > follow;  
  
    // FIRST(A) = {a, e}  
    first["A"].insert('a');  
    first["A"].insert('#');  
  
    // FIRST(B) = {b, c}  
    first["B"].insert('b');  
    first["B"].insert('c');  
  
    // FIRST(S) = {a, b, c} since A can derive epsilon  
    first["S"].insert('a');  
    first["S"].insert('b');  
    first["S"].insert('c');  
  
    // FOLLOW(S) = {$}  
    follow["S"].insert('$');  
  
    // FOLLOW(A) = FIRST(B) = {b, c}  
    follow["A"].insert('b');
```

```

follow["A"].insert('c');

// FOLLOW(B) = FOLLOW(S) = {$}

follow["B"].insert('$');

cout << "FIRST SETS:\n";

printSet("FIRST(S)", first["S"]);
printSet("FIRST(A)", first["A"]);
printSet("FIRST(B)", first["B"]);

cout << "\nFOLLOW SETS:\n";

printSet("FOLLOW(S)", follow["S"]);
printSet("FOLLOW(A)", follow["A"]);
printSet("FOLLOW(B)", follow["B"]);

return 0;

}

```

Output:

```

GRAMMAR 1:
S -> A B
A -> a A | e
B -> b B | c

FIRST SETS:
FIRST(S) = { a b c }
FIRST(A) = { e a }
FIRST(B) = { b c }

FOLLOW SETS:
FOLLOW(S) = { $ }
FOLLOW(A) = { b c }
FOLLOW(B) = { $ }

-----
Process exited after 0.2733 seconds with return value 0
Press any key to continue . . .

```

Task 2:

$E \rightarrow T R$

$R \rightarrow + T R \mid \varepsilon$

$T \rightarrow F Y$

$Y \rightarrow * F Y \mid \varepsilon$

$F \rightarrow (E) \mid id$

$First:-$
 $First(F) = \{ (, id \}$
 $First(Y) = \{ +, E \}$
 $First(T) = \{ (, id \}$
 $First(R) = \{ +, E \}$
 $First(E) = \{ (, id \}$
 $Follow:-$
 $Follow(E) = \{), \$ \}$
 $Follow(R) = \{), \$ \}$
 $Follow(T) = \{ +,), \$ \}$
 $Follow(Y) = \{ +,), \$ \}$
 $Follow(F) = \{ +, +,), \$ \}$

Code:

// Task2.cpp - Simple FIRST and FOLLOW for Grammar 2

```
#include <iostream>
```

```
#include <map>
```

```
#include <set>
```

```
#include <string>
```

```
using namespace std;
```

```
void printSet(string name, set<char> s) {
```

```
    cout << name << " = { ";
```

```
    for (set<char>::iterator it = s.begin(); it != s.end(); ++it) {
```

```
        if (*it == '#') cout << "e ";
```

```
        else cout << *it << " ";
```

```
    }
```

```
    cout << "}\n";
```

```
}
```

```
int main() {
```

```
    cout << "GRAMMAR 2:\n";
```

```
    cout << "E -> T R\n";
```

```
cout << "R -> + T R | e\n";  
cout << "T -> F Y\n";  
cout << "Y -> * F Y | e\n";  
cout << "F -> ( E ) | id\n\n";
```

```
map<string, set<char> > first;
```

```
// FIRST(F) = {(, i}
```

```
first["F"].insert('(');
```

```
first["F"].insert('i');
```

```
// FIRST(Y) = {*, e}
```

```
first["Y"].insert('*');
```

```
first["Y"].insert('#');
```

```
// FIRST(T) = FIRST(F) = {(, i}
```

```
first["T"].insert('(');
```

```
first["T"].insert('i');
```

```
// FIRST(R) = {+, e}
```

```
first["R"].insert('+');
```

```
first["R"].insert('#');
```

```
// FIRST(E) = FIRST(T) = {(, i}
```

```
first["E"].insert('(');
```

```
first["E"].insert('i');
```

```
map<string, set<char> > follow;
```

```
// FOLLOW(E) = {$, )}
```

```
follow["E"].insert('$');
```

```
follow["E"].insert(')');
```

```

// FOLLOW(R) = FOLLOW(E) = { $, ) }
follow["R"].insert('$');
follow["R"].insert('');
// FOLLOW(T) = FIRST(R) - { e } u FOLLOW(E) = { +, $, ) }
follow["T"].insert('+');
follow["T"].insert('$');
follow["T"].insert('');
// FOLLOW(Y) = FOLLOW(T) = { +, $, ) }
follow["Y"].insert('+');
follow["Y"].insert('$');
follow["Y"].insert('');
// FOLLOW(F) = FIRST(Y) - { e } u FOLLOW(T) = { *, +, $, ) }
follow["F"].insert('*');
follow["F"].insert('+');
follow["F"].insert('$');
follow["F"].insert('');
cout << "FIRST SETS:\n";
printSet("FIRST(E)", first["E"]);
printSet("FIRST(R)", first["R"]);
printSet("FIRST(T)", first["T"]);
printSet("FIRST(Y)", first["Y"]);
printSet("FIRST(F)", first["F"]);
cout << "\nFOLLOW SETS:\n";
printSet("FOLLOW(E)", follow["E"]);
printSet("FOLLOW(R)", follow["R"]);
printSet("FOLLOW(T)", follow["T"]);
printSet("FOLLOW(Y)", follow["Y"]);
printSet("FOLLOW(F)", follow["F"]);
return 0;

```

}

Output:

```
GRAMMAR 2:
E -> T R
R -> + T R | e
T -> F Y
Y -> * F Y | e
F -> ( E ) | id

FIRST SETS:
FIRST(E) = { ( i }
FIRST(R) = { e + }
FIRST(T) = { ( i }
FIRST(Y) = { e * }
FIRST(F) = { ( i }

FOLLOW SETS:
FOLLOW(E) = { $ ) }
FOLLOW(R) = { $ ) }
FOLLOW(T) = { $ ) + }
FOLLOW(Y) = { $ ) + }
FOLLOW(F) = { $ ) * + }

-----
Process exited after 0.3145 seconds with return value 0
Press any key to continue . . .
```

Task 3:

$S \rightarrow A B C$

$A \rightarrow a \mid \epsilon$

$B \rightarrow b \mid \epsilon$

$C \rightarrow c$

First:-
First(A) = {a, ε}
First(B) = {b, ε}
First(C) = {c}
First(S) = {a, b, c}
Follow:-
Follow(S) = {\$}
Follow(A) = {b, c}
Follow(B) = {ε}
Follow(C) = {ε}

Code:

// Task3.cpp - Simple FIRST and FOLLOW for Grammar 3

```

#include <iostream>

#include <map>

#include <set>

#include <string>

using namespace std;

void printSet(string name, set<char> s) {

    cout << name << " = { ";

    for (set<char>::iterator it = s.begin(); it != s.end(); ++it) {

        if (*it == '#') cout << "e ";

        else cout << *it << " ";

    }

    cout << "}\n";

}

int main() {

    cout << "GRAMMAR 3:\n";

    cout << "S -> A B C\n";

    cout << "A -> a | e\n";

    cout << "B -> b | e\n";

    cout << "C -> c\n\n";

    map<string, set<char> > first;

    // FIRST(A) = {a, e}

    first["A"].insert('a');

    first["A"].insert('#');

    // FIRST(B) = {b, e}

    first["B"].insert('b');

    first["B"].insert('#');

    // FIRST(C) = {c}

```

```

first["C"].insert('c');

// FIRST(S) = {a, b, c} since A and B can derive epsilon
first["S"].insert('a');
first["S"].insert('b');
first["S"].insert('c');

map<string, set<char> > follow;

// FOLLOW(S) = {$}
follow["S"].insert('$');

// FOLLOW(A) = FIRST(B) - {e} u FIRST(C) = {b, c}
follow["A"].insert('b');
follow["A"].insert('c');

// FOLLOW(B) = FIRST(C) u FOLLOW(S) = {c, $}
follow["B"].insert('c');
follow["B"].insert('$');

// FOLLOW(C) = FOLLOW(S) = {$}
follow["C"].insert('$');

cout << "FIRST SETS:\n";
printSet("FIRST(S)", first["S"]);
printSet("FIRST(A)", first["A"]);
printSet("FIRST(B)", first["B"]);
printSet("FIRST(C)", first["C"]);

cout << "\nFOLLOW SETS:\n";
printSet("FOLLOW(S)", follow["S"]);
printSet("FOLLOW(A)", follow["A"]);
printSet("FOLLOW(B)", follow["B"]);

```

```

    printSet("FOLLOW(C)", follow["C"]);

    return 0;
}

```

Output:

```

GRAMMAR 3:
S -> A B C
A -> a | e
B -> b | e
C -> c

FIRST SETS:
FIRST(S) = { a b c }
FIRST(A) = { e a }
FIRST(B) = { e b }
FIRST(C) = { c }

FOLLOW SETS:
FOLLOW(S) = { $ }
FOLLOW(A) = { b c }
FOLLOW(B) = { $ c }
FOLLOW(C) = { $ }

-----
Process exited after 0.3697 seconds with return value 0
Press any key to continue . . .

```

Task 4:

$S \rightarrow a S \mid b A$

$A \rightarrow c A \mid d$

First :-

First (A) = { c, d }

First (S) = { a, b }

Follow :-

Follow (S) = { \$ }

Follow (A) = { \$ }

Code:

```

// Task4.cpp - Simple FIRST and FOLLOW for Grammar 4

#include <iostream>

#include <map>

```

```

#include <set>

#include <string>

using namespace std;

void printSet(string name, set<char> s) {
    cout << name << " = { ";
    for (set<char>::iterator it = s.begin(); it != s.end(); ++it) {
        cout << *it << " ";
    }
    cout << "}\n";
}

```

```

int main() {
    cout << "GRAMMAR 4:\n";
    cout << "S -> a S | b A\n";
    cout << "A -> c A | d\n\n";

    map<string, set<char> > first;

    // FIRST(A) = {c, d}
    first["A"].insert('c');
    first["A"].insert('d');

    // FIRST(S) = {a, b}
    first["S"].insert('a');
    first["S"].insert('b');
    map<string, set<char> > follow;

    // FOLLOW(S) = {$}
    follow["S"].insert('$');
}

```

```

// FOLLOW(A) = FOLLOW(S) = {$}

follow["A"].insert('$');

cout << "FIRST SETS:\n";

printSet("FIRST(S)", first["S"]);
printSet("FIRST(A)", first["A"]);

cout << "\nFOLLOW SETS:\n";

printSet("FOLLOW(S)", follow["S"]);
printSet("FOLLOW(A)", follow["A"]);

return 0;
}

```

Output:

```

GRAMMAR 4:
S -> a S | b A
A -> c A | d

FIRST SETS:
FIRST(S) = { a b }
FIRST(A) = { c d }

FOLLOW SETS:
FOLLOW(S) = { $ }
FOLLOW(A) = { $ }

```

Task 5:

$S \rightarrow (S)S \mid \epsilon$

First :-
 $\text{First}(S) = \{ (, \epsilon \}$
 Follow:-
 $\text{Follow}(S) = \{), \$ \}$

Code:

```
// Task5.cpp - Simple FIRST and FOLLOW for Grammar 5

#include <iostream>

#include <map>

#include <set>

#include <string>

using namespace std;

void printSet(string name, set<char> s) {

    cout << name << " = { ";

    for (set<char>::iterator it = s.begin(); it != s.end(); ++it) {

        if (*it == '#') cout << "e ";

        else cout << *it << " ";

    }

    cout << "}\n";

}

int main() {

    cout << "GRAMMAR 5:\n";

    cout << "S -> ( S ) S | e\n\n";

    map<string, set<char> > first;

    // FIRST(S) = {(, e}

    first["S"].insert('(');

    first["S"].insert('#');

    map<string, set<char> > follow;

    // FOLLOW(S) = {$, )}

    // 1. Start symbol gets $

    // 2. From S -> ( S ) S, first S is followed by ) so ) is in FOLLOW(S)

    // 3. Second S is at end so FOLLOW(S) includes FOLLOW(S) itself

    follow["S"].insert('$');

    follow["S"].insert(')');

    cout << "FIRST SETS:\n";

    printSet("FIRST(S)", first["S"]);
```

```

cout << "\nFOLLOW SETS:\n";

printSet("FOLLOW(S)", follow["S"]);

return 0;
}

```

Output:

```

S -> ( S ) S | e

FIRST SETS:
FIRST(S) = { e ( }

FOLLOW SETS:
FOLLOW(S) = { $ ) }

-----
Process exited after 0.3552 seconds with return value 0
Press any key to continue . . .

```

Task 6:

$S \rightarrow AB$

$A \rightarrow a \mid \epsilon$

$B \rightarrow bC$

$C \rightarrow c \mid \epsilon$

First :-

First(A) = { a, ϵ }.

First(C) = { c, ϵ }

First(B) = { b }.

First(S) = { a, b }.

Follow :-

Follow(S) = { \$ }.

Follow(A) = { b }.

Follow(B) = { \$ }.

Follow(C) = { \$ }.

Code:

```
// Task6.cpp - Simple FIRST and FOLLOW for Grammar 6
```

```
#include <iostream>
```

```
#include <map>
```

```
#include <set>
```

```
#include <string>
```

```
using namespace std;
```

```
void printSet(string name, set<char> s) {
```

```
    cout << name << " = { ";
```

```
    for (set<char>::iterator it = s.begin(); it != s.end(); ++it) {
```

```
        if (*it == '#') cout << "e ";
```

```
        else cout << *it << " ";
```

```
    }
```

```
    cout << "}\n";
```

```
}
```

```
int main() {
```

```
    cout << "GRAMMAR 6:\n";
```

```
    cout << "S -> A B\n";
```

```
    cout << "A -> a | e\n";
```

```
    cout << "B -> b C\n";
```

```
    cout << "C -> c | e\n\n";
```

```
    map<string, set<char> > first;
```

```
    // FIRST(A) = {a, e}
```

```
    first["A"].insert('a');
```

```
    first["A"].insert('#');
```

```

// FIRST(C) = {c, e}

first["C"].insert('c');
first["C"].insert('#');


// FIRST(B) = {b}

first["B"].insert('b');

// FIRST(S) = FIRST(A) - {e} u FIRST(B) = {a, b}

first["S"].insert('a');
first["S"].insert('b');

map<string, set<char> > follow;

// FOLLOW(S) = {$}

follow["S"].insert('$');

// FOLLOW(A) = FIRST(B) = {b}

follow["A"].insert('b');

// FOLLOW(B) = FOLLOW(S) = {$}

follow["B"].insert('$');

// FOLLOW(C) = FOLLOW(B) = {$}

follow["C"].insert('$');

cout << "FIRST SETS:\n";

printSet("FIRST(S)", first["S"]);
printSet("FIRST(A)", first["A"]);
printSet("FIRST(B)", first["B"]);
printSet("FIRST(C)", first["C"]);

cout << "\nFOLLOW SETS:\n";

printSet("FOLLOW(S)", follow["S"]);
printSet("FOLLOW(A)", follow["A"]);
printSet("FOLLOW(B)", follow["B"]);
printSet("FOLLOW(C)", follow["C"]);

return 0;

```

}

Output:

```
S -> A B
A -> a | e
B -> b C
C -> c | e

FIRST SETS:
FIRST(S) = { a b }
FIRST(A) = { e a }
FIRST(B) = { b }
FIRST(C) = { e c }

FOLLOW SETS:
FOLLOW(S) = { $ }
FOLLOW(A) = { b }
FOLLOW(B) = { $ }
FOLLOW(C) = { $ }

-----
Process exited after 0.3403 seconds with return value 0
Press any key to continue . . .
```

Task 7:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow id \mid (E)$

Firste-
First (E) = { id, (}
First (E') = { +, ε }
First (T) = { id, (}
First (T') = { +, ε }
First (E) = { id, (}
Follows-
Follow (E) = {), \$ }
Follow (E') = {), \$ }
Follow (T) = { +,), \$ }
Follow (T') = { +,), \$ }
Follow (F) = { +,), \$ }

Code:

```
// Task7.cpp - Simple FIRST and FOLLOW for Grammar 7
```

```
#include <iostream>
```

```
#include <map>
```

```
#include <set>
```

```
#include <string>
```

```
using namespace std;
```

```
void printSet(string name, set<char> s) {
```

```
    cout << name << " = { ";
```

```
    for (set<char>::iterator it = s.begin(); it != s.end(); ++it) {
```

```
        if (*it == '#') cout << "e ";
```

```
        else cout << *it << " ";
```

```
    }
```

```
    cout << "}\n";
```

```
}
```

```
int main() {
```

```
    cout << "GRAMMAR 7:\n";
```

```
    cout << "E -> T E'\n";
```

```
    cout << "E' -> + T E' | e'\n";
```

```
    cout << "T -> F T'\n";
```

```
    cout << "T' -> * F T' | e'\n";
```

```
    cout << "F -> id | ( E )\n\n";
```

```
    map<string, set<char> > first;
```

```
    // FIRST(F) = {(, i}
```

```
    first["F"].insert('(');
```

```
    first["F"].insert('i');
```

```
    // FIRST(T) = {*, e}
```

```
    first["T"].insert('*');
```

```
    first["T"].insert('#');
```

```

// FIRST(T) = FIRST(F) = {(, i}
first["T"].insert('(');
first["T"].insert('i');
// FIRST(E') = {+, e}
first["E'"].insert('+');
first["E'"].insert('#');
// FIRST(E) = FIRST(T) = {(, i}
first["E"].insert('(');
first["E"].insert('i');
map<string, set<char> > follow;
// FOLLOW(E) = {$, )}
follow["E"].insert('$');
follow["E"].insert(')');
// FOLLOW(E') = FOLLOW(E) = {$, )}
follow["E'"].insert('$');
follow["E'"].insert(')');
// FOLLOW(T) = FIRST(E')-{e} u FOLLOW(E) = {+, $, )}
follow["T"].insert('+');
follow["T"].insert('$');
follow["T"].insert(')');
// FOLLOW(T') = FOLLOW(T) = {+, $, )}
follow["T'"].insert('+');
follow["T'"].insert('$');
follow["T'"].insert(')');
// FOLLOW(F) = FIRST(T')-{e} u FOLLOW(T) = {*, +, $, )}
follow["F"].insert('*');
follow["F"].insert('+');
follow["F"].insert('$');
follow["F"].insert(')');

```

```

cout << "FIRST SETS:\n";

printSet("FIRST(E) ", first["E"]);
printSet("FIRST(E')", first["E'"]);
printSet("FIRST(T) ", first["T"]);
printSet("FIRST(T')", first["T'"]);
printSet("FIRST(F) ", first["F"]);

cout << "\nFOLLOW SETS:\n";

printSet("FOLLOW(E) ", follow["E"]);
printSet("FOLLOW(E')", follow["E'"]);
printSet("FOLLOW(T) ", follow["T"]);
printSet("FOLLOW(T')", follow["T'"]);
printSet("FOLLOW(F) ", follow["F"]);

return 0;
}

```

Output:

```

E  -> T E'
E' -> + T E' | e
T  -> F T'
T' -> * F T' | e
F  -> id | ( E )

FIRST SETS:
FIRST(E)  = { ( i }
FIRST(E') = { e + }
FIRST(T)  = { ( i }
FIRST(T') = { e * }
FIRST(F)  = { ( i }

FOLLOW SETS:
FOLLOW(E)  = { $ ) }
FOLLOW(E') = { $ ) }
FOLLOW(T)  = { $ ) + }
FOLLOW(T') = { $ ) + }
FOLLOW(F)  = { $ ) * + }

-----
Process exited after 0.3534 seconds with return value 0
Press any key to continue . . .

```

Task 8:

$S \rightarrow A B$

$A \rightarrow B a \mid \epsilon$

$B \rightarrow b \mid \epsilon$

First:-
 $\text{First}(B) = \{ b, \epsilon \}$
 $\text{First}(A) = \{ b, a, \epsilon \}$
Follow:-
 $\text{Follow}(S) = \{ \$ \}$
 $\text{Follow}(A) = \{ b, \$ \}$
 $\text{Follow}(B) = \{ a, \$ \}$

Code:

// Task8.cpp - Simple FIRST and FOLLOW for Grammar 8

```
#include <iostream>
```

```
#include <map>
```

```
#include <set>
```

```
#include <string>
```

```
using namespace std;
```

```
void printSet(string name, set<char> s) {
```

```
    cout << name << " = { ";
```

```
    for (set<char>::iterator it = s.begin(); it != s.end(); ++it) {
```

```
        if (*it == '#') cout << "e ";
```

```
        else cout << *it << " ";
```

```
    }
```

```
    cout << "}\n";
```

```
}
```

```

int main() {

    cout << "GRAMMAR 8:\n";

    cout << "S -> A B\n";

    cout << "A -> B a | e\n";

    cout << "B -> b | e\n\n";


    map<string, set<char> > first;


    // FIRST(B) = {b, e}

    first["B"].insert('b');

    first["B"].insert('#');


    // FIRST(A) = FIRST(B) u {a} since A -> B a and B can be e

    first["A"].insert('b');

    first["A"].insert('a');

    first["A"].insert('#');


    // FIRST(S) = FIRST(A)-{e} u FIRST(B) since A can be e

    first["S"].insert('b');

    first["S"].insert('a');

    map<string, set<char> > follow;

    // FOLLOW(S) = {$}

    follow["S"].insert('$');

    // FOLLOW(A) = FIRST(B)-{e} u FOLLOW(S) since B can be e

    // = {b} u {$}

    follow["A"].insert('b');

    follow["A"].insert('$');


    // FOLLOW(B):

```

```

// From A -> B a : {a} is in FOLLOW(B)

// From S -> A B : FOLLOW(S) = {$} is in FOLLOW(B)

// From S -> A B : since B is last, FOLLOW(S) included

// From A -> B a : a is right after B

follow["B"].insert('a');

follow["B"].insert('b');

follow["B"].insert('$');

cout << "FIRST SETS:\n";

printSet("FIRST(S)", first["S"]);

printSet("FIRST(A)", first["A"]);

printSet("FIRST(B)", first["B"]);

cout << "\nFOLLOW SETS:\n";

printSet("FOLLOW(S)", follow["S"]);

printSet("FOLLOW(A)", follow["A"]);

printSet("FOLLOW(B)", follow["B"]);

return 0;

}

```

Output:

```

S -> A B
A -> B a | e
B -> b | e

FIRST SETS:
FIRST(S) = { a b }
FIRST(A) = { e a b }
FIRST(B) = { e b }

FOLLOW SETS:
FOLLOW(S) = { $ }
FOLLOW(A) = { $ b }
FOLLOW(B) = { $ a b }

```

Task 9:

$S \rightarrow A B C$

$A \rightarrow a A \mid \epsilon$

$B \rightarrow b B \mid \epsilon$

$C \rightarrow c C \mid d$

First:-
First(A) = {a, ε}
First(B) = {b, ε}
First(C) = {c, d}
First(s) = {a, b, c, d}
Follow:-
Follow(s) = { \$ }
Follow(A) = { b, c, d }
Follow(B) = { c, d }
Follow(C) = { \$ }.

Code:

// Task9.cpp - Simple FIRST and FOLLOW for Grammar 9

```
#include <iostream>
```

```
#include <map>
```

```
#include <set>
```

```
#include <string>
```

```
using namespace std;
```

```
void printSet(string name, set<char> s) {
```

```
    cout << name << " = { ";
```

```
    for (set<char>::iterator it = s.begin(); it != s.end(); ++it) {
```

```
        if (*it == '#') cout << "e ";
```

```

        else cout << *it << " ";
    }
    cout << "}\n";
}

```

```

int main() {
    cout << "GRAMMAR 9:\n";
    cout << "S -> A B C\n";
    cout << "A -> a A | e\n";
    cout << "B -> b B | e\n";
    cout << "C -> c C | d\n\n";

    map<string, set<char> > first;

    // FIRST(A) = {a, e}
    first["A"].insert('a');
    first["A"].insert('#');

    // FIRST(B) = {b, e}
    first["B"].insert('b');
    first["B"].insert('#');

    // FIRST(C) = {c, d}
    first["C"].insert('c');
    first["C"].insert('d');

    // FIRST(S) = {a, b, c, d} since A and B can derive epsilon
    first["S"].insert('a');
    first["S"].insert('b');

```

```

first["S"].insert('c');
first["S"].insert('d');
map<string, set<char> > follow;
// FOLLOW(S) = {$}
follow["S"].insert('$');
// FOLLOW(A) = FIRST(B)-{e} u FIRST(C) = {b, c, d}
follow["A"].insert('b');
follow["A"].insert('c');
follow["A"].insert('d');
// FOLLOW(B) = FIRST(C) u FOLLOW(S) = {c, d, $}
follow["B"].insert('c');
follow["B"].insert('d');
follow["B"].insert('$');
// FOLLOW(C) = FOLLOW(S) = {$}
follow["C"].insert('$');
cout << "FIRST SETS:\n";
printSet("FIRST(S)", first["S"]);
printSet("FIRST(A)", first["A"]);
printSet("FIRST(B)", first["B"]);
printSet("FIRST(C)", first["C"]);
cout << "\nFOLLOW SETS:\n";
printSet("FOLLOW(S)", follow["S"]);
printSet("FOLLOW(A)", follow["A"]);
printSet("FOLLOW(B)", follow["B"]);
printSet("FOLLOW(C)", follow["C"]);
return 0;
}

```

Output:

```

S → A B C
A → a A | ε
B → b B | ε
C → c C | d

```

FIRST SETS:

FIRST(S) = { a b c d }

FIRST(A) = { ε a }

FIRST(B) = { ε b }

FIRST(C) = { c d }

FOLLOW SETS:

FOLLOW(S) = { \$ }

FOLLOW(A) = { b c d }

FOLLOW(B) = { \$ c d }

FOLLOW(C) = { \$ }

Task 10:

$S \rightarrow A C B$

$A \rightarrow a A \mid \epsilon$

$B \rightarrow b B \mid \epsilon$

$C \rightarrow c \mid d$

Firsts -

First(A) = { a, ε }

First(B) = { ε, b }

First(C) = { c, d }

First(S) = { a, c, d }

Follows -

Follow(S) = { \$ }

Follow(A) = { c, d }

Follow(C) = { b, \$ }

Follow(B) = { \$ }

Code:

// Task10.cpp - Simple FIRST and FOLLOW for Grammar 10

```

#include <iostream>

#include <map>

#include <set>

#include <string>

using namespace std;

void printSet(string name, set<char> s) {

    cout << name << " = { ";

    for (set<char>::iterator it = s.begin(); it != s.end(); ++it) {

        if (*it == '#') cout << "e ";

        else cout << *it << " ";

    }

    cout << "}\n";

}

int main() {

    cout << "GRAMMAR 10:\n";

    cout << "S -> A C B\n";

    cout << "A -> a A | e\n";

    cout << "B -> b B | e\n";

    cout << "C -> c | d\n\n";


    map<string, set<char> > first;


    // FIRST(A) = {a, e}

    first["A"].insert('a');

    first["A"].insert('#');


    // FIRST(B) = {b, e}

    first["B"].insert('b');

    first["B"].insert('#');

```

```

// FIRST(C) = {c, d}

first["C"].insert('c');
first["C"].insert('d');


// FIRST(S) = FIRST(A)-{e} u FIRST(C) u FIRST(B)-{e} = {a, c, d, b}

first["S"].insert('a');
first["S"].insert('c');
first["S"].insert('d');
first["S"].insert('b');


map<string, set<char> > follow;


// FOLLOW(S) = {$}

follow["S"].insert('$');


// FOLLOW(A) = FIRST(C) = {c, d}

follow["A"].insert('c');
follow["A"].insert('d');

// FOLLOW(C) = FIRST(B)-{e} u FOLLOW(S) = {b, $}

follow["C"].insert('b');
follow["C"].insert('$');

// FOLLOW(B) = FOLLOW(S) = {$}

follow["B"].insert('$');

cout << "FIRST SETS:\n";

printSet("FIRST(S) ", first["S"]);
printSet("FIRST(A) ", first["A"]);
printSet("FIRST(B) ", first["B"]);
printSet("FIRST(C) ", first["C"]);

```

```
cout << "\nFOLLOW SETS:\n";

printSet("FOLLOW(S) ", follow["S"]);
printSet("FOLLOW(A) ", follow["A"]);
printSet("FOLLOW(B) ", follow["B"]);
printSet("FOLLOW(C) ", follow["C"]);

return 0;
}
```

Output:

```
S -> A C B
A -> a A | e
B -> b B | e
C -> c | d

FIRST SETS:
FIRST(S)  = { a b c d }
FIRST(A)  = { e a }
FIRST(B)  = { e b }
FIRST(C)  = { c d }

FOLLOW SETS:
FOLLOW(S) = { $ }
FOLLOW(A) = { c d }
FOLLOW(B) = { $ }
FOLLOW(C) = { $ b }
```